



南開大學
Nankai University

计算机学院
人工智能导论实验报告

实验三：垃圾短信识别

姓名：林盛森

学号：2312631

专业：计算机科学与技术

2025 年 4 月 18 日

目录

1 问题重述	2
1.1 实验背景	2
1.2 实验要求	2
1.3 实验环境	2
2 设计思想	2
2.1 数据预处理	2
2.2 构建训练集和验证集	3
2.3 模型训练与测试	3
2.4 保存训练好的模型	4
3 代码内容	4
3.1 train.py	4
3.2 main.py	6
4 实验结果	6
4.1 各项指标	6
4.2 样例测试	7
5 总结	7

1 问题重述

1.1 实验背景

垃圾短信 (Spam Messages, SM) 是指未经过用户同意向用户发送不愿接收的商业广告或者不符合法律规范的短信。

随着手机的普及, 垃圾短信在日常生活日益泛滥, 已经严重的影响到了人们的正常生活娱乐, 乃至社会的稳定。

据 360 公司 2020 年第一季度有关手机安全的报告提到, 360 手机卫士在第一季度共拦截各类垃圾短信约 34.4 亿条, 平均每日拦截垃圾短信约 3784.7 万条。

大数据时代的到来使得大量个人信息数据得以沉淀和积累, 但是庞大的数据量缺乏有效的整理规范; 在面对量级如此巨大的短信数据时, 为了保证更良好的用户体验, 如何从数据中挖掘出更多有意义的信息为人们免受垃圾短信骚扰成为当前亟待解决的问题。

1.2 实验要求

任务提供包括数据读取、基础模型、模型训练等基本代码

参赛选手需完成核心模型构建代码, 并尽可能将模型调到最佳状态

模型单次推理时间不超过 10 秒

1.3 实验环境

可以使用基于 Python 的 Pandas、Numpy、Sklearn 等库进行相关特征处理, 使用 Sklearn 框架训练分类器, 也可编写深度学习模型, 使用过程中请注意 Python 包 (库) 的版本。

2 设计思想

这次实验的主要目的就是让我们基于已给的数据集去训练模型, 这是一个分类任务, 我们需要依次进行数据的预处理、训练集和验证集的划分、基于训练集训练模型、基于验证集验证模型, 并在此基础上不断优化模型。

2.1 数据预处理

首先导入相关的包, 接着利用 pd 中读文件的功能去读取对应地址下的数据集, 在之前的训练的过程中, 我们已可以得出样本不平衡会对结果造成影响, 所以我们需要从样本数量比较大的一类中提取样本, 采样到相同数量, 使得两种标签的样本数量平衡。

```
1 # 导入相关的包
2 import warnings
3 warnings.filterwarnings('ignore')
4 import os
5 os.environ["HDF5_USE_FILE_LOCKING"] = "FALSE"
6 import pandas as pd
7 import numpy as np
8
9 # 数据集的路径
```

```
10 data_path = "./datasets/5f9ae242cae5285cd734b91e-momodel/sms_pub.csv"
11 # 读取数据
12 sms = pd.read_csv(data_path, encoding='utf-8')
13 sms_pos = sms[(sms['label'] == 1)]
14 sms_neg = sms[(sms['label'] == 0)].sample(frac=1.0)[: len(sms_pos)]
15 sms = pd.concat([sms_pos, sms_neg], axis=0).sample(frac=1.0)#采样到相同数量
```

接着读停用词并构建停用词表，每一行都是一个停用词。

```
1 stopwords_path = r'scu_stopwords.txt'
2 def read_stopwords(stopwords_path):
3     stopwords = []
4     with open(stopwords_path, 'r', encoding='utf-8') as f:
5         stopwords = f.read()
6         stopwords = stopwords.splitlines()
7     return stopwords
8
9 # 读取停用词
10 stopwords = read_stopwords(stopwords_path)
```

2.2 构建训练集和验证集

然后对样本进行划分，0.9 为训练集，0.1 为验证集。

```
1 # 构建训练集和测试集
2 from sklearn.model_selection import train_test_split, GridSearchCV
3 X = np.array(sms.msg_new)
4 y = np.array(sms.label)
5 X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42,
6     test_size=0.1)
7 print("总共的数据大小", X.shape)
8 print("训练集数据大小", X_train.shape)
9 print("测试集数据大小", X_test.shape)
```

2.3 模型训练与测试

接着进入核心部分，训练模型。通过构建 pipeline，使用归一化参数 MaxAbsScaler，并设置 ngram_range 为 (1,2) 增强上下文语义关联性，并利用朴素贝叶斯模型去训练样本。

```
1 # ----- 导入相关的库 -----
2 from sklearn.pipeline import Pipeline
3 from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
4 from sklearn.naive_bayes import BernoulliNB
5 from sklearn.naive_bayes import MultinomialNB
6 from sklearn.preprocessing import MaxAbsScaler
7 # pipeline_list 用于传给 Pipeline 作为参数
8 pipeline_list = [
```

```

9      ('tfidf',
      TfidfVectorizer(stop_words=stopwords, ngram_range=(1,2), sublinear_tf=True)),
10     ('scaler', MaxAbsScaler()), #归一化
11     ('classifier', MultinomialNB())
12 ]
13 pipeline=Pipeline(pipeline_list)
14 pipeline.fit(X_train, y_train)

```

然后就可以基于验证集去验证训练的效果如何，打印出相关信息以便于我们调整训练参数。

```

1 y_pred = pipeline.predict(X_test)
2
3 # 在测试集上进行评估
4 from sklearn import metrics
5 print("在测试集上的混淆矩阵：")
6 print(metrics.confusion_matrix(y_test, y_pred))
7 print("在测试集上的分类结果报告：")
8 print(metrics.classification_report(y_test, y_pred))
9 print("在测试集上的 f1-score：")
10 print(metrics.f1_score(y_test, y_pred))

```

2.4 保存训练好的模型

最后把训练的模型保存下来。

```

1 # 在所有的样本上训练一次，充分利用已有的数据，提高模型的泛化能力
2 pipeline.fit(X, y)
3 # 保存训练的模型，请将模型保存在 results 目录下
4 from sklearn.externals import joblib
5 pipeline_path = 'results/pipeline0.model'
6 joblib.dump(pipeline, pipeline_path)

```

3 代码内容

3.1 train.py

```

1 # 导入相关的包
2 import warnings
3 warnings.filterwarnings('ignore')
4 import os
5 os.environ["HDF5_USE_FILE_LOCKING"] = "FALSE"
6 import pandas as pd
7 import numpy as np
8
9 # 数据集的路径
10 data_path = './datasets/5f9ae242cae5285cd734b91e-momodel/sms_pub.csv'
11 # 读取数据

```

```

12 sms = pd.read_csv(data_path, encoding='utf-8')
13 sms_pos = sms[(sms['label'] == 1)]
14 sms_neg = sms[(sms['label'] == 0)].sample(frac=1.0)[: len(sms_pos)]
15 sms = pd.concat([sms_pos, sms_neg], axis=0).sample(frac=1.0)#采样到相同数量
16
17 stopwords_path = r'scu_stopwords.txt'
18 def read_stopwords(stopwords_path):
19     stopwords = []
20     with open(stopwords_path, 'r', encoding='utf-8') as f:
21         stopwords = f.read()
22     stopwords = stopwords.splitlines()
23     return stopwords
24
25 # 读取停用词
26 stopwords = read_stopwords(stopwords_path)
27
28 # 构建训练集和测试集
29 from sklearn.model_selection import train_test_split, GridSearchCV
30 X = np.array(sms.msg_new)
31 y = np.array(sms.label)
32 X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42,
33     test_size=0.1)
34 print("总共的数据大小", X.shape)
35 print("训练集数据大小", X_train.shape)
36 print("测试集数据大小", X_test.shape)
37
38 from sklearn.pipeline import Pipeline
39 from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
40 from sklearn.naive_bayes import BernoulliNB
41 from sklearn.naive_bayes import MultinomialNB
42 from sklearn.preprocessing import MaxAbsScaler
43 # pipeline_list 用于传给 Pipeline 作为参数
44 pipeline_list = [
45     ('tfidf',
46      TfidfVectorizer(stop_words=stopwords, ngram_range=(1,2), sublinear_tf=True)),
47     ('scaler', MaxAbsScaler()), # 归一化
48     ('classifier', MultinomialNB())
49 ]
50 pipeline = Pipeline(pipeline_list)
51 pipeline.fit(X_train, y_train)
52 y_pred = pipeline.predict(X_test)
53
54 # 在测试集上进行评估
55 from sklearn import metrics
56 print("在测试集上的混淆矩阵: ")
57 print(metrics.confusion_matrix(y_test, y_pred))
58 print("在测试集上的分类结果报告: ")
59 print(metrics.classification_report(y_test, y_pred))
60 print("在测试集上的 f1-score : ")

```

```

59 print(metrics.f1_score(y_test, y_pred))
60
61 # 在所有的样本上训练一次，充分利用已有的数据，提高模型的泛化能力
62 pipeline.fit(X, y)
63 # 保存训练的模型，请将模型保存在 results 目录下
64 from sklearn.externals import joblib
65 pipeline_path = 'results/pipeline0.model'
66 joblib.dump(pipeline, pipeline_path)

```

3.2 main.py

```

1 # 加载训练好的模型
2 from sklearn.externals import joblib
3 # ----- pipeline 保存的路径，若有变化请修改 -----
4 pipeline_path = 'results/pipeline0.model'
5 pipeline = joblib.load(pipeline_path)
6
7 def predict(message):
8
9     label = pipeline.predict([message])[0]
10    proba = list(pipeline.predict_proba([message])[0])
11
12    return label, proba

```

4 实验结果

4.1 各项指标

在测试集上的混淆矩阵：

```
[[7353  528]
 [  47 7902]]
```

在测试集上的分类结果报告：

	precision	recall	f1-score	support
0	0.99	0.93	0.96	7881
1	0.94	0.99	0.96	7949
accuracy			0.96	15830
macro avg	0.97	0.96	0.96	15830
weighted avg	0.97	0.96	0.96	15830

在测试集上的 f1-score :

0.9648940716771476

图 4.1: 各项指标

可以看到，对于 0/1 标签来说都有比较高的精确率，而且 f1-score 和 accuracy 的值都比较高。并且在混淆矩阵中，被错误划分的数据也相应变少。

4.2 样例测试

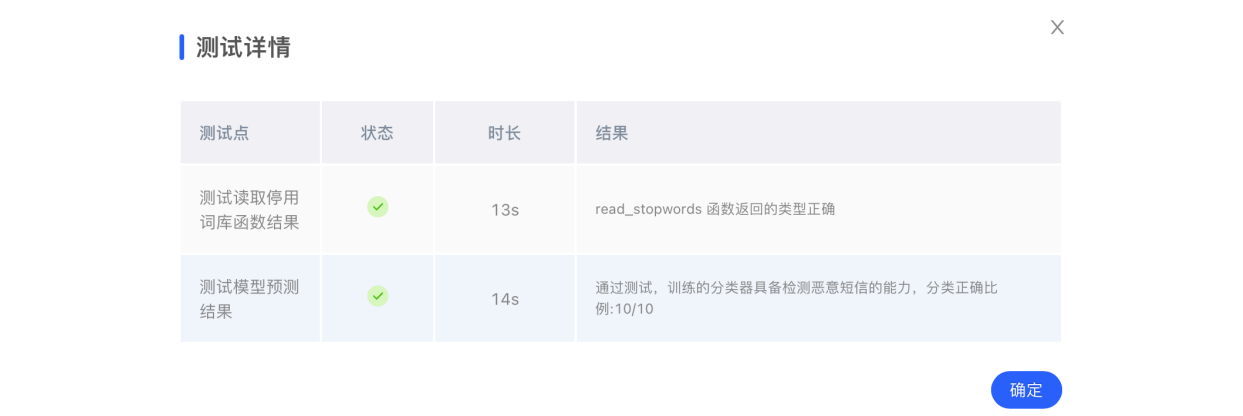


图 4.2: 样例测试结果

接着进行样例测试，可以看到十个样例全部测试正确。

5 总结

本实验基于朴素贝叶斯模型，我们通过调整参数，以得到更好的模型，重点注意的是需要让两种标签的样本数量达到平衡。通过本实验，完成了数据的预处理、模型的训练、模型的优化等操作，实现了垃圾信息识别的功能。

通过本实验，我对监督学习有了更好的理解。